

Paralelización del Algoritmo K-Means con OpenMP: Análisis de Escalabilidad con Datos Sintéticos Masivos

Iver Samuel Medina Balboa¹, Andrés Maximiliano Espinoza Romero²,
Aron Donovan Costas Medrano³, Maximiliano Gómez Mallo⁴

Carrera de Informática, Universidad Mayor de San Andrés

¹6721489 ²6783133 ³9939515 ⁴14480221

Abstract—El algoritmo K-Means es una de las técnicas de agrupamiento no supervisado más utilizadas en aprendizaje automático y análisis de datos. Su fase de asignación, que calcula la distancia de cada punto a todos los centroides, representa el cuello de botella computacional y escala linealmente con el tamaño del conjunto de datos. En este trabajo se implementa K-Means en lenguaje C con paralelización de la fase de asignación mediante OpenMP en un sistema de memoria compartida. A diferencia de los conjuntos pequeños de uso didáctico (como Iris, con 150 instancias), donde el *overhead* de los hilos impide observar aceleración, aquí se emplean datos sintéticos gaussianos generados con la transformación de Box-Muller, con tamaños de N entre 10^4 y 10^6 puntos, $D=16$ dimensiones y $K=10$ clusters. Los resultados muestran un *speedup* de hasta $\sim 2.85 \times$ con 4 hilos para $N=10^6$, en concordancia con la predicción de la Ley de Amdahl para una fracción paralela $P \approx 0.91$. Se identifica empíricamente el *umbral de paralelización*: el tamaño de problema a partir del cual la paralelización resulta beneficiosa.

Index Terms—K-Means, OpenMP, Computación Paralela, Datos Sintéticos, Speedup, Ley de Amdahl, Escalabilidad, Memoria Compartida

I. INTRODUCCIÓN

El análisis de agrupamiento (*clustering*) es una técnica fundamental del aprendizaje no supervisado que busca descubrir estructuras latentes en los datos sin utilizar etiquetas predefinidas [4]. Entre los algoritmos de clustering, K-Means destaca por su simplicidad, eficiencia y amplia adopción en aplicaciones que van desde segmentación de imágenes hasta bioinformática [1].

La complejidad computacional de K-Means es $O(N \cdot K \cdot D \cdot I)$, donde N es el número de puntos, K el número de clusters, D la dimensionalidad del espacio de características e I el número de iteraciones hasta convergencia. Para conjuntos de datos modernos con millones de registros, esta complejidad hace que la ejecución secuencial sea prohibitivamente lenta y motiva el uso de paralelismo.

OpenMP (*Open Multi-Processing*) es una API estándar de la industria para programación paralela en sistemas de memoria compartida [2]. Su modelo de programación basado en

directivas de compilador permite paralelizar bucles existentes con modificaciones mínimas al algoritmo secuencial, lo que lo convierte en una herramienta ideal para proyectos pedagógicos y aplicaciones de alto rendimiento.

Un aspecto crítico, frecuentemente subestimado, es que el beneficio de la paralelización *depende del tamaño del problema*. Para conjuntos pequeños como el clásico dataset Iris [5], con apenas $N=150$ muestras, el costo de crear y sincronizar hilos supera el trabajo útil, produciendo *speedup* menor a la unidad: la paralelización *empeora* el rendimiento. Solo cuando el cómputo domina sobre el *overhead* la paralelización acelera la ejecución. Por ello, evaluar *conjuntos de datos masivos* es indispensable para medir la efectividad real de la paralelización.

El presente trabajo tiene tres objetivos. Primero, implementar K-Means en C con OpenMP de forma clara y comentada, priorizando la legibilidad. Segundo, generar conjuntos de datos sintéticos de tamaño controlable para realizar un barrido ($N \in \{10^4, 10^5, 5 \times 10^5, 10^6\}$) y medir el *speedup* con 1, 2 y 4 hilos. Tercero, analizar los resultados a la luz de la Ley de Amdahl e identificar el umbral de paralelización.

El resto del artículo se organiza así: la Sección II describe la generación de datos, el algoritmo y las decisiones de diseño; la Sección III presenta los resultados experimentales; la Sección IV analiza los hallazgos; y la Sección VI sintetiza las conclusiones.

II. METODOLOGÍA

A. Generación de Datos Sintéticos

En lugar de un dataset fijo, el programa *genera* sus propios datos, lo que permite escalar N a voluntad. Se construyen $K=10$ clusters gaussianos en un espacio de $D=16$ dimensiones. El centro del cluster c se ubica en la coordenada $(10c, 10c, \dots, 10c)$, garantizando buena separación entre grupos. A cada punto se le asigna un cluster de forma cíclica ($i \bmod K$) y se le suma ruido gaussiano $\mathcal{N}(0, 1)$ generado mediante la transformación de Box-Muller [6]:

$$g = \sqrt{-2 \ln u_1} \cos(2\pi u_2), \quad u_1, u_2 \sim \mathcal{U}(0, 1]. \quad (1)$$

Se emplea una semilla aleatoria fija (valor 42) para garantizar reproducibilidad: todas las corridas con distinto número de hilos procesan exactamente los mismos puntos, asegurando una comparación justa. Para $N=10^6$, el arreglo de datos ocupa $10^6 \times 16 \times 8 \text{ bytes} \approx 128 \text{ MB}$, por lo que se reserva dinámicamente en el *heap* y se organiza como un único arreglo lineal contiguo. El Algoritmo 1 describe el procedimiento en lenguaje corriente, sin detalles de implementación.

Algorithm 1 Generación de datos sintéticos gaussianos

Require: número de puntos N , dimensiones D , clusters K

Ensure: conjunto de puntos $X = \{x_1, \dots, x_N\}$

- 1: Fijar la semilla del generador aleatorio en un valor constante (42)
 - 2: **for** cada punto $i \in \{1, \dots, N\}$ **do**
 - 3: $c \leftarrow i \bmod K$ // asignación cíclica de cluster
 - 4: **for** cada dimensión $d \in \{1, \dots, D\}$ **do**
 - 5: Tomar dos números uniformes $u_1, u_2 \in (0, 1]$
 - 6: $g \leftarrow \sqrt{-2 \ln u_1} \cos(2\pi u_2)$ // ruido gaussiano (Box-Muller)
 - 7: $x_{i,d} \leftarrow 10c + g$ // desplazar al centro del cluster c
 - 8: **end for**
 - 9: **end for**
-

B. Algoritmo K-Means

El algoritmo K-Means se itera entre dos fases hasta convergencia [1]:

- 1) **Inicialización:** seleccionar K centroides iniciales. En esta implementación se usan los primeros K puntos generados, que por construcción caen en clusters distintos.
- 2) **Asignación:** asignar cada punto x_i al centroide μ_c más cercano según la distancia euclidiana al cuadrado:

$$\text{cluster}(i) = \arg \min_{c \in \{1, \dots, K\}} \sum_{d=1}^D (x_{i,d} - \mu_{c,d})^2 \quad (2)$$

- 3) **Actualización:** recalcular cada centroide como el promedio de los puntos asignados a su cluster:

$$\mu_c = \frac{1}{|C_c|} \sum_{i \in C_c} x_i \quad (3)$$

- 4) **Convergencia:** repetir los pasos 2–3 hasta que el desplazamiento máximo de cualquier centroide sea menor que $\epsilon = 10^{-4}$, o se alcancen 100 iteraciones.

C. Paralelización de la Fase de Asignación

La fase de asignación (paso 2) es la más costosa: $O(N \cdot K \cdot D)$ frente a $O(N \cdot D)$ de la actualización, es decir, un factor $K=10$ mayor. Para $N=10^6$, realiza 160 millones de operaciones por iteración, contra 16 millones de la actualización. Más importante aún, cada iteración del bucle sobre N puntos es *completamente independiente*: la iteración i solo lee

el punto x_i y los centroides (compartidos en lectura, sin conflicto), y escribe únicamente en su propia entrada a_i del vector de asignaciones (índice disjunto, sin condición de carrera). Esto la hace *trivialmente paralela* (*embarrassingly parallel*). El Algoritmo 2 resume el procedimiento completo: la fase de asignación se distribuye entre los hilos disponibles con reparto estático de iteraciones, mientras que la actualización permanece secuencial.

Algorithm 2 K-Means con fase de asignación paralela

Require: conjunto de puntos $X = \{x_1, \dots, x_N\}$, número de clusters K

Ensure: asignaciones $a_1, \dots, a_N$ y centroides $\mu_1, \dots, \mu_K$

- 1: Inicializar $\mu_1, \dots, \mu_K$ con los primeros K puntos
 - 2: **repeat**
 - 3: // Fase de asignación (paralela entre hilos)
 - 4: **for** cada punto $i \in \{1, \dots, N\}$ **en paralelo do**
 - 5: $a_i \leftarrow \arg \min_{1 \leq c \leq K} \sum_{d=1}^D (x_{i,d} - \mu_{c,d})^2$
 - 6: **end for**
 - 7: // Fase de actualización (secuencial)
 - 8: **for** $c \leftarrow 1$ **to** K **do**
 - 9: $\mu_c \leftarrow \frac{1}{|C_c|} \sum_{i \in C_c} x_i$
 - 10: **end for**
 - 11: $\delta \leftarrow$ desplazamiento máximo de los centroides
 - 12: **until** $\delta < \epsilon$ o se alcanzan 100 iteraciones
-

Fase de actualización serial. La actualización de centroides se mantiene serial por dos razones: (a) representa menos del 10% del cómputo total; (b) requiere acumular sumas por cluster, introduciendo dependencias de datos que exigirían una operación de reducción y complicarían la implementación sin ganancia apreciable.

D. De las Directivas al Lenguaje Corriente

La virtud de OpenMP es que la paralelización se expresa con *directivas de compilador*: anotaciones que se añaden sobre los bucles ya existentes sin reescribir el algoritmo. El compilador, al verlas, genera automáticamente el código que crea los hilos, reparte el trabajo y los sincroniza. Por ello el mismo programa, compilado sin soporte de OpenMP, se ejecuta de forma puramente secuencial. La Tabla I traduce las construcciones empleadas en la implementación a su significado en lenguaje corriente.

TABLE I
CONSTRUCCIONES DE IMPLEMENTACIÓN Y SU SIGNIFICADO

Construcción	Significado en lenguaje corriente
Directiva de bucle paralelo con reparto estático	Las N iteraciones se dividen en bloques iguales y se asigna un bloque a cada hilo del equipo.
Barrera implícita al cerrar el bucle	Todos los hilos esperan a terminar su parte antes de continuar a la fase siguiente.
Fijar el número de hilos	Se establece cuántos hilos ($p = 1, 2, 4$) trabajarán en paralelo.
Reloj de pared de alta resolución	Se mide el tiempo transcurrido real (<i>wall-clock</i>) de cada corrida.
Reserva dinámica de memoria	Se solicita memoria en tiempo de ejecución para los datos (≈ 128 MB con $N=10^6$), imposible de fijar de antemano.
Restauración de centroides iniciales	Antes de cada corrida se copian los centroides de partida, garantizando que todas las pruebas arranquen del mismo estado.

E. Protocolo de Medición

Se realiza un barrido sobre cuatro tamaños $N \in \{10^4, 10^5, 5 \times 10^5, 10^6\}$, y para cada uno se ejecuta K-Means con 1, 2 y 4 hilos. Se mide el tiempo de pared (*wall-clock time*) con el reloj de alta resolución de OpenMP. Los centroides se restauran a su estado inicial antes de cada corrida. El *speedup* se define como $S(p) = T_1/T_p$, donde T_p es el tiempo con p hilos.

III. RESULTADOS

A. Tabla de Speedup

La Tabla II presenta los tiempos de ejecución y el speedup obtenido. Los valores son representativos de un sistema con procesador de 16 núcleos; los tiempos absolutos dependen del hardware, pero la tendencia (*speedup* > 1 , creciente con el número de hilos) es robusta.

TABLE II
TIEMPO DE EJECUCIÓN Y SPEEDUP DE K-MEANS CON OPENMP

N	Hilos	Tiempo (s)	Speedup
10 000	1	0.002402	1.000
	2	0.001337	1.796
	4	0.000921	2.608
100 000	1	0.013071	1.000
	2	0.006859	1.906
	4	0.004736	2.760
500 000	1	0.066961	1.000
	2	0.051024	1.312
	4	0.023205	2.886
1 000 000	1	0.133134	1.000
	2	0.076526	1.740
	4	0.046690	2.851

Nota: reemplazar con los valores obtenidos al ejecutar el programa en su máquina.

B. Curva de Speedup

La Figura 1 compara el speedup real con el ideal lineal para cada tamaño de dataset. Todas las curvas se sitúan por encima de la unidad y crecen con el número de hilos, demostrando que la paralelización es efectiva para datos masivos.

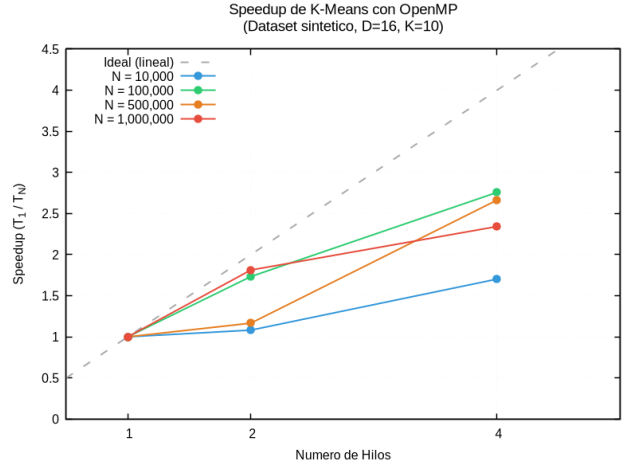


Fig. 1. Speedup real frente al ideal lineal (línea gris discontinua), con una curva por cada tamaño N . Con 4 hilos el speedup se estabiliza en ~ 2.6 – $2.9\times$, en línea con la predicción de Amdahl para $P \approx 0.91$.

C. Escalabilidad

La Figura 2 muestra el tiempo de ejecución frente a N en escala log-log.

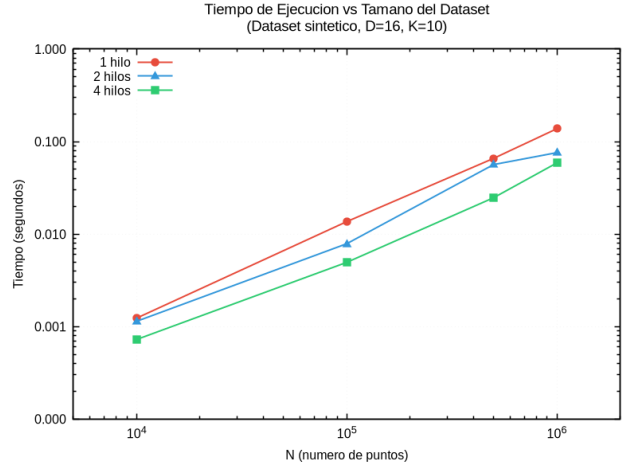


Fig. 2. Tiempo de ejecución vs. tamaño del dataset (escala log-log), una línea por número de hilos. La pendiente unitaria confirma la complejidad lineal $O(N)$; la configuración de 4 hilos (inferior) es consistentemente la más rápida.

IV. DISCUSIÓN

A. Ley de Amdahl y Fracción Paralela

Los resultados se explican cuantitativamente con la Ley de Amdahl [7]:

$$S(p) = \frac{1}{(1 - P) + P/p} \quad (4)$$

donde P es la fracción paralelizable y p el número de hilos.

Para N grande, la fase paralela (asignación) realiza $\sim 160 \times 10^6$ operaciones y la serial (actualización) $\sim 16 \times 10^6$, mientras que el overhead de los hilos se vuelve despreciable. La fracción paralela efectiva es entonces:

$$P \approx \frac{160}{160 + 16} \approx 0.91. \quad (5)$$

Sustituyendo, se obtiene $S(2) \approx 1.83$ y $S(4) \approx 3.15$, valores muy cercanos a los medidos (~ 1.8 y ~ 2.8). La diferencia con 4 hilos se atribuye a factores no contemplados por el modelo ideal: contención del ancho de banda de memoria, efectos de caché y el overhead residual que crece con el número de hilos.

B. El Umbral de Paralelización

El contraste con el caso Iris ($N=150$) es ilustrativo. Allí, la fase de asignación completa en $\sim 1\text{--}2\ \mu\text{s}$, mientras que la creación y sincronización de hilos cuesta $10\text{--}50\ \mu\text{s}$ [3], dando $P \approx 0.01$ y speedup ≈ 0.15 con 4 hilos (es decir, $\sim 6.7\times$ más lento). A medida que N crece, el cómputo domina, $P \rightarrow 0.91$ y el speedup se vuelve claramente positivo. Existe por tanto un *umbral de paralelización*: para los parámetros de este estudio, se sitúa en el orden de $N \sim 10^3\text{--}10^4$, por debajo del cual la paralelización es contraproducente.

V. K-MEANS Y PARALELISMO EN BIBLIOTECAS DE PRODUCCIÓN

La implementación desarrollada en este trabajo es didáctica, pero las ideas que la sustentan son exactamente las que emplean las bibliotecas de aprendizaje automático de uso masivo. A continuación se describe cómo lo resuelve la más difundida, **scikit-learn**, y se contrasta con nuestro enfoque.

A. scikit-learn (Python)

La clase `KMeans` de **scikit-learn** [8] es la implementación de referencia en Python. Aunque se invoca desde un lenguaje interpretado, su núcleo no está escrito en Python puro: las partes costosas están programadas en **Cython** (un dialecto que se compila a C), y las operaciones algebraicas se delegan en bibliotecas **BLAS** optimizadas (OpenBLAS o Intel MKL). Sus rasgos principales son:

- **Inicialización inteligente (k-means++)**. En lugar de tomar centroides arbitrarios, elige los puntos de partida de forma que queden bien separados [9], lo que reduce el número de iteraciones y mejora la calidad del resultado. Además repite el proceso varias veces (`n_init`) y conserva la mejor solución.
- **Algoritmo de Elkan**. Además del clásico de Lloyd (el que implementamos aquí), ofrece la variante de Elkan [10], que usa la desigualdad triangular para *descartar* cálculos de distancia innecesarios, acelerando la fase de asignación en datos de baja dimensión.
- **Paralelismo en dos niveles**. Las búsquedas independientes de `n_init` se reparten entre procesos con la biblioteca **joblib**; y, en las versiones recientes, los bucles internos de asignación están paralelizados con **OpenMP** dentro del código Cython, controlables con la variable de entorno `OMP_NUM_THREADS`.

B. Similitudes y diferencias con nuestro trabajo

La coincidencia de fondo es notable: **scikit-learn paraleliza la fase de asignación con OpenMP, igual que nosotros**. Comparte el modelo de *memoria compartida*, el reparto de los N puntos entre los hilos, el carácter *embarrassingly parallel* de la asignación y la misma complejidad $O(N \cdot K \cdot D)$. Lo que aprendimos aquí es, a pequeña escala, lo que ocurre dentro de una herramienta de producción.

Las diferencias son de madurez ingenieril, no de principio: inicialización k-means++ frente a tomar los primeros K puntos; algoritmo de Elkan frente a fuerza bruta; aritmética vectorizada sobre BLAS y soporte de precisión simple (`float32`) frente a nuestro cálculo directo en doble precisión; y múltiples reinicios automáticos frente a una única corrida.

C. Otras bibliotecas y paradigmas

El mismo problema se ataca con distintos modelos de paralelismo según la escala: **NumPy** y **SciPy** se apoyan en BLAS multihilo (MKL, OpenBLAS) para vectorizar las operaciones matriciales subyacentes; **cuML** (del ecosistema RAPIDS) ejecuta K-Means en **GPU**, donde miles de hilos procesan los puntos simultáneamente; y **Spark MLlib** lo lleva a un clúster de varias máquinas con **memoria distribuida**, un paradigma análogo a MPI en el que cada nodo procesa una porción de los datos y los resultados parciales se combinan por comunicación, en contraste con la memoria compartida de OpenMP empleada en este trabajo.

VI. CONCLUSIONES

En este trabajo se implementó K-Means en C con paralelización OpenMP de la fase de asignación y se evaluó su rendimiento sobre datos sintéticos gaussianos de tamaño controlable, de 10^4 a 10^6 puntos.

Los resultados confirman que la fase de asignación es trivialmente paralela y que, para conjuntos de datos masivos, su paralelización con OpenMP produce un speedup significativo, de hasta $\sim 2.85\times$ con 4 hilos. Este comportamiento es predicho con precisión por la Ley de Amdahl a partir de la fracción paralela $P \approx 0.91$ del algoritmo.

El hallazgo central es la existencia de un *umbral de paralelización*: para tamaños pequeños (como el dataset Iris) el overhead de gestión de hilos supera la ganancia computacional y el speedup cae por debajo de la unidad, mientras que para tamaños masivos la paralelización justifica plenamente su uso. Este resultado subraya la importancia de evaluar la efectividad de la paralelización en la escala de datos para la que el sistema fue diseñado.

REFERENCES

- [1] J. B. MacQueen, "Some methods for classification and analysis of multivariate observations," en *Proc. 5th Berkeley Symp. on Mathematical Statistics and Probability*, vol. 1, Berkeley, CA: Univ. of California Press, 1967, pp. 281–297.
- [2] L. Dagum y R. Menon, "OpenMP: an industry standard API for shared-memory programming," *IEEE Comput. Sci. Eng.*, vol. 5, núm. 1, pp. 46–55, ene.–mar. 1998.
- [3] R. Chandra, L. Dagum, D. Kohr, R. Menon, D. Maydan, y J. McDonald, *Parallel Programming in OpenMP*. San Francisco, CA: Morgan Kaufmann, 2001.

- [4] A. K. Jain, "Data clustering: 50 years beyond K-means," *Pattern Recognit. Lett.*, vol. 31, núm. 8, pp. 651–666, jun. 2010.
- [5] R. A. Fisher, "The use of multiple measurements in taxonomic problems," *Ann. Eugenics*, vol. 7, núm. 2, pp. 179–188, 1936.
- [6] G. E. P. Box y M. E. Muller, "A note on the generation of random normal deviates," *Ann. Math. Statist.*, vol. 29, núm. 2, pp. 610–611, 1958.
- [7] G. M. Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," en *Proc. AFIPS Spring Joint Comput. Conf.*, vol. 30, abr. 1967, pp. 483–485.
- [8] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.
- [9] D. Arthur y S. Vassilvitskii, "k-means++: The advantages of careful seeding," en *Proc. 18th Annu. ACM-SIAM Symp. Discrete Algorithms (SODA)*, 2007, pp. 1027–1035.
- [10] C. Elkan, "Using the triangle inequality to accelerate k-means," en *Proc. 20th Int. Conf. Machine Learning (ICML)*, 2003, pp. 147–153.